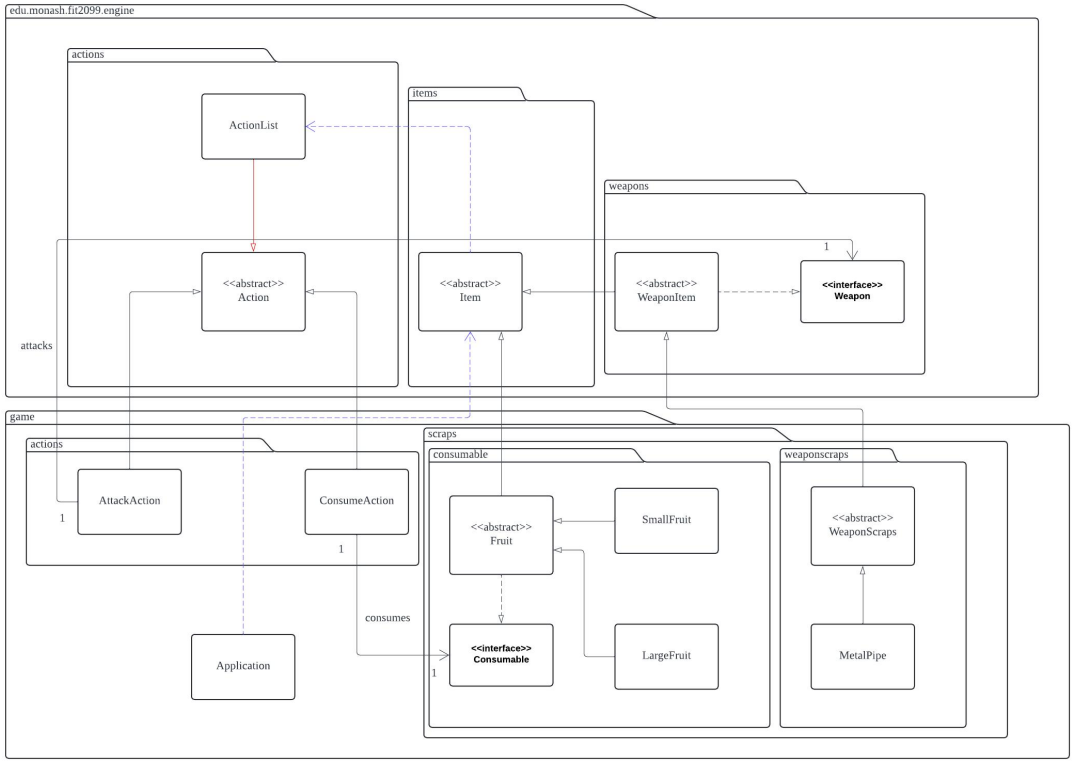


This diagram represents object-oriented design of Assignment 1 Requirement 4 which aims to introduce scraps with different functionality such as being a weapon or able to heal the player. The diagram consists of 2 main packages, the game engine provided and the game itself. Inside the engine package, the abstract classes involved are Action, Item and WeaponItem. The concrete class involved is ActionList and the interface involved is Weapon.

Inside the game package, the two action concrete classes involved for this requirement 4 are AttackAction and ConsumeAction which both inherits from the Action abstract class in the engine package. Compared to requirement 2, there is also an introduction of interface Consumable which is implemented by the Fruit abstract class that is inherited by LargeFruit and SmallFruit concrete classes. Lastly, we have a new abstract class WeaponScraps which is inherited by concrete class MetalPipe. There is also an Application class inside the game package.

The object-oriented design principles applied in this design are:

- Single Responsibility Principle
- Open-Closed Principle
- Dependency Inversion Principle
- Liskov Substitution Principle
- DRY Principle

How are the principles being applied?

- Each classes focus on a specific aspect on the game. For example, AttackAction is an action which represents an action of attacking of one entity to another and ConsumeAction is an action which represents one entity consuming consumable scraps. This design allows each class to bear a single responsibility which the code can be easy to maintain. (Single Responsibility Principle)

- The design allows extension without modifying the existing code. If a new type of weapon scrap or consumable item is introduced, the concrete implementations can be simply added by extending using the abstract classes without altering existing code. This promotes code extensibility and flexibility. (Open-Closed Principle)

- The design encourages usage of abstractions rather than direct concrete implementations which reduce coupling between the classes and promotes code maintainability and readability. It allows generic characteristics among the concrete classes being shared without displaying direct dependencies between them. (Dependency Inversion Principle)

- Polymorphism can be applied in the design which allows an uniform handling towards different types of entities in the game. For example, a Fruit object and a WeaponScraps object can be treated as an Item object. This design enables a consistency in behaviour across different types of entities in the game. (Liskov Substitution Principle)

- Instead of implementing the same characteristics within the same concrete classes, the design relies on abstraction which clears redundant code and promotes code reusability. (DRY Principle)

Let's extend the design by introducing a new type of item to the game such as MedicalPill. This class represents an item medical pill which appears on the ground that can be consumed by the player, but it will hurt the player by 3 HP. Since this is not a type of Fruit, it does not inherit from the Fruit abstract class, meaning it is an Item class which implements the Consumable interface. After that, we have to implement the consumedEffect() from the Consumable interface which reduces the player health by 3 points when it is consumed.

When the Application runs, the MedicalPill object will appear in the preset coordinate. When the player pick the MedicalPill object up by using the PickUpAction before, the player is given a new option of consuming the MedicalPill the next turn. Once the player consumes the MedicalPill, the object is removed from the player's inventory and the player health decreased by 3 points.

Based on the description of the potential extension above, this OOP design is shown to seamlessly integrate the new features by utilising the inheritance and interface implementation. There is a minimal implementation for this extension as it only involves the creation of MedicalPill item class and implements the consumedEffect() from the Consumable interface implemented. This design provides flexibility as it is easy to extend the code for new features. It provides a robust base for the game to continue develop on.